Name - Aayush Patil
Div - D20A
Roll No - 56

Experiment No: 3

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

```
!pip install flask requests
```

```
Requirement already satisfied: flask in /usr/local/lib/python3.13/dist-packages (3.1.3)
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (2.32.4
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.13/dist-packages (
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.13/dist-packages (fr
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.13/dist-packa
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.13/dist-packages (f
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.13/dist-package
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (fr
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packag
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packag
```

Create the Blockchain class

```
class Blockchain:

    def __init__(self, node_address):
        self.chain = []
        self.transactions = []
        self.nodes = set()
        self.node_address = node_address

        # Create Genesis Block
        self.create_block(proof=1, previous_hash='0')


    # ------------------------------------------------
    # CREATE BLOCK
    # ------------------------------------------------

    def create_block(self, proof, previous_hash):

        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions.copy()
```

Create the 3 blockchain nodes

```
blockchains = {
    5001: Blockchain(5001),
    5002: Blockchain(5002),
    5003: Blockchain(5003)
}
```

```
for port, blockchain in blockchains.items():
    print(
        f"Node {port} -> "
        f"Blockchain length: {len(blockchain.chain)}"
    )
```

```
Node 5001 -> Blockchain length: 1
Node 5002 -> Blockchain length: 1
Node 5003 -> Blockchain length: 1
```

Create Flask APIs

```
def create_app(port):

    app = Flask(__name__)

    blockchain = blockchains[port]


    # ================================================
    # CONNECT NODE
    # ================================================

    @app.route('/connect_node', methods=['POST'])
    def connect_node():

        json_data = request.get_json()

        nodes = json_data.get('nodes')

        if not nodes:
            return jsonify({
```

```
# ================================================
# ADD TRANSACTION
# ================================================

@app.route('/add_transaction', methods=['POST'])
def add_transaction():

    json_data = request.get_json()

    sender = json_data['sender']
    receiver = json_data['receiver']
    amount = json_data['amount']

    index = blockchain.add_transaction(
        sender,
        receiver,
        amount
    )

    return jsonify({
        'message': f'Transaction will be added to Block {index}'
    }), 201
```

Start all 3 nodes

```
apps = {}

for port in [5001, 5002, 5003]:

    app = create_app(port)

    apps[port] = app

    thread = threading.Thread(
        target=app.run,
        kwargs={
            'host': '0.0.0.0',
            'port': port,
            'debug': False,
            'use_reloader': False
        }
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deploym
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://172.28.0.12:5001
INFO:werkzeug:Press CTRL+C to quit
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deploym
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://172.28.0.12:5002
INFO:werkzeug:Press CTRL+C to quit
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deploym
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5003
 * Running on http://172.28.0.12:5003
INFO:werkzeug:Press CTRL+C to quit
All 3 blockchain nodes started!
```

Test Node 5001

```
response = requests.get(
    'http://127.0.0.1:5001/get_chain'
)

print(response.json())
```

```
3.0.1 - - [27/Aug/2026 03:25:35] "GET /get_chain HTTP/1.1" 200 -
': 1, 'previous_hash': '0', 'proof': 1, 'timestamp': '2026-08-27 03:23:34.800999', 'transac
```

Connect the three nodes

```
requests.post(
    'http://127.0.0.1:5001/connect_node',
    json={
        'nodes': [
            'http://127.0.0.1:5002',
            'http://127.0.0.1:5003'
        ]
    }
).json()
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:26:22] "POST /connect_node HTTP/1.1" 201 -
{'message': 'All the nodes are now connected.',
 'total_nodes': ['127.0.0.1:5002', '127.0.0.1:5003']}
```

```
requests.post(
    'http://127.0.0.1:5002/connect_node',
    json={
        'nodes': [
            'http://127.0.0.1:5001',
            'http://127.0.0.1:5003'
        ]
    }
).json()
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:26:37] "POST /connect_node HTTP/1.1" 201
{'message': 'All the nodes are now connected.',
 'total_nodes': ['127.0.0.1:5003', '127.0.0.1:5001']}
```

```
requests.post(
    'http://127.0.0.1:5003/connect_node',
    json={
        'nodes': [
            'http://127.0.0.1:5001',
            'http://127.0.0.1:5002'
        ]
    }
).json()
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:26:46] "POST /connect_node HTTP/1.1" 20
{'message': 'All the nodes are now connected.',
 'total_nodes': ['127.0.0.1:5002', '127.0.0.1:5001']}
```

Verify connections

```
for port in [5001, 5002, 5003]:

    print(f"\nNODE {port}")

    print(
        blockchains[port].nodes
    )
```

```
NODE 5001
{'127.0.0.1:5002', '127.0.0.1:5003'}

NODE 5002
{'127.0.0.1:5003', '127.0.0.1:5001'}

NODE 5003
{'127.0.0.1:5002', '127.0.0.1:5001'}
```

Add transactions

```
{
    "sender": "a",
    "receiver": "b",
    "amount": 5
}
```

```
{'sender': 'a', 'receiver': 'b', 'amount': 5}
```

Transaction 1

```
requests.post(
    'http://127.0.0.1:5001/add_transaction',
    json={
        'sender': 'Aayush',
        'receiver': 'Nikhil Kale',
        'amount': 5
    }
).json()
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 04:20:01] "POST /add_transaction HTTP/1.1" 201 -
{'message': 'Transaction will be added to Block 3'}
```

Transaction 2

```
requests.post(
    'http://127.0.0.1:5001/add_transaction',
    json={
        'sender': 'Aayush1',
        'receiver': 'Patil',
        'amount': 10
    }
).json()
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 04:19:58] "POST /add_transaction HTTP/1.1" 201 -
{'message': 'Transaction will be added to Block 3'}
```

Transaction 3

```
requests.post(
    'http://127.0.0.1:5001/add_transaction',
    json={
        'sender': 'Yogita ',
        'receiver': 'Aayush2',
        'amount': 15
    }
).json()
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 04:19:50] "POST /add_transaction HTTP/1.1" 201 -
{'message': 'Transaction will be added to Block 3'}
```

Check pending transactions

```
print(blockchains[5001].transactions)
```

```
[{'sender': 'Yogita ', 'receiver': 'Aayush2', 'amount': 15}]
```

Check chain lengths BEFORE mining

```
for port in [5001, 5002, 5003]:

    response = requests.get(
        f'http://127.0.0.1:{port}/get_chain'
    )

    print(
        f"Node {port}: "
        f"{response.json()['length']} blocks"
    )
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:30:06] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:30:06] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:30:06] "GET /get_chain HTTP/1.1" 200 -
Node 5001: 1 blocks
Node 5002: 1 blocks
Node 5003: 1 blocks
```

Mine a block on Node 5001

```
response = requests.get(
    'http://127.0.0.1:5001/mine_block'
)

result = response.json()

print(json.dumps(result, indent=4))
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 04:22:02] "GET /mine_block HTTP/1.1" 20
{
    "index": 3,
    "message": "Congratulations, you just mined a block!",
    "previous_hash": "07aef36cd97b7bfae54aad577d0909d8964e6b2b9c784fad40a487f7534688b7",
    "proof": 45293,
    "timestamp": "2026-08-27 04:22:02.740165",
    "transactions": [
        {
            "amount": 15,
            "receiver": "Aayush2",
            "sender": "Yogita "
        },
        {
            "amount": 10,
            "receiver": "Patil",
            "sender": "Aayush1"
        },
        {
            "amount": 5,
            "receiver": "Nikhil Kale",
            "sender": "Aayush"
        },
        {
            "amount": 1,
            "receiver": "Miner_5001",
            "sender": "6fbf651b-635c-44bc-92ef-341719127fc6"
        }
```

Check all three chain lengths

```
for port in [5001, 5002, 5003]:

    response = requests.get(
        f'http://127.0.0.1:{port}/get_chain'
    )

    print(
        f"Node {port}: "
        f"{response.json()['length']} blocks"
    )
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:30:54] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:30:54] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:30:54] "GET /get_chain HTTP/1.1" 200 -
Node 5001: 2 blocks
Node 5002: 1 blocks
Node 5003: 1 blocks
```

Demonstrate replace_chain()

Now Node 5002 knows about Node 5001 but doesn't have its longer blockchain.

```
response = requests.get(
    'http://127.0.0.1:5002/replace_chain'
)

print(json.dumps(response.json(), indent=4))
```

```
{
    "index": 3,
    "previous_hash": "07aef36cd97b7bfae54aad577d0909d8964e6b2b9c784fad40a487f753468b7",
    "proof": 45293,
    "timestamp": "2026-08-27 04:22:02.740165",
    "transactions": [
        {
            "amount": 15,
            "receiver": "Aayush2",
            "sender": "Yogita "
        },
        {
            "amount": 10,
            "receiver": "Patil",
            "sender": "Aayush1"
        },
        {
            "amount": 5,
            "receiver": "Nikhil Kale",
            "sender": "Aayush"
        },
        {
            "amount": 1,
            "receiver": "Miner_5001",
            "sender": "6fbf651b-635c-44bc-92ef-341719127fc6"
```


Check Node 5002

```
response = requests.get(
    'http://127.0.0.1:5002/get_chain'
)

print(
    "Node 5002 chain length:",
    response.json()['length']
)
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:32:49] "GET /get_chain HTTP/1.1" 200 -
Node 5002 chain length: 2
```

Replace chain on Node 5003

```
response = requests.get(
    'http://127.0.0.1:5003/replace_chain'
)

print(json.dumps(response.json(), indent=4))
```


```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:33:02] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:33:02] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:33:02] "GET /replace_chain HTTP/1.1" 200 -
{
    "message": "The chain was replaced by the longest chain.",
    "new_chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-08-27 03:23:34.800999",
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "d30a08af6a2cfda3addb4718bb702b73c542695c4b45967d05b4de58cc0a95f1",
            "proof": 533,
            "timestamp": "2026-08-27 03:30:22.882331",
            "transactions": [
```

Final verification

```
for port in [5001, 5002, 5003]:

    response = requests.get(
        f'http://127.0.0.1:{port}/get_chain'
    )

    data = response.json()

    print(
        f"Node {port} → "
        f"{data['length']} blocks"
    )
```

```
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:33:59] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:33:59] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [27/Aug/2026 03:33:59] "GET /get_chain HTTP/1.1" 200 -
Node 5001 → 2 blocks
Node 5002 → 2 blocks
Node 5003 → 2 blocks
```

transactions were removed

```
print(
    "Pending transactions on Node 5001:"
)

print(
    blockchains[5001].transactions
)
```

```
Pending transactions on Node 5001:
[]
```

Check the complete chain

```
response = requests.get(
    'http://127.0.0.1:5001/get_chain'
)

data = response.json()

print(json.dumps(data, indent=4))
```


```
    {
        "index": 3,
        "previous_hash": "07aef36cd97b7bfae54aad577d0909d8964e6b2b9c784fad40a487f7534688b7",
        "proof": 45293,
        "timestamp": "2026-08-27 04:22:02.740165",
        "transactions": [
            {
                "amount": 15,
                "receiver": "Aayush2",
                "sender": "Yogita "
            },
            {
                "amount": 10,
                "receiver": "Patil",
                "sender": "Aayush1"
            },
            {
                "amount": 5,
                "receiver": "Nikhil Kale",
                "sender": "Aayush"
            },
            {
                "amount": 1,
                "receiver": "Miner_5001",
                "sender": "6fbf651b-635c-44bc-92ef-341719127fc6"
            }
        ]
    }
],
"length": 3
```

Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.